

Software Engineering -M3-A1

Section A — Concept Application

S1. Waterfall vs Agile Methodology

Agile methodology is recommended for this project because the requirements change frequently and the client wants to review working features every two weeks. Agile divides the project into small iterations called sprints, allowing the team to develop, test, deliver, and receive feedback regularly. This makes it easier to respond to changing requirements.

If the wrong methodology is chosen, two risks are:

1. **Difficulty handling changing requirements:** Changes may become expensive and time-consuming.
2. **Delayed feedback and delivery:** The client may not be able to review working features frequently, which can lead to misunderstandings and rework.

S2. Git Workflow

The team should follow a **branch-based Git workflow**. The main branch should contain only stable and tested code. Each teammate should create a separate feature branch for their work instead of directly modifying the main branch.

After completing the work, the developer should commit and push the changes to the feature branch. A pull request should then be created so another teammate can review and test the code before it is merged into the main branch.

Important Git commands include:

- **git checkout -b** — creates a new branch for separate development.
- **git add** — prepares modified files for committing.
- **git commit** — records the changes with a meaningful message.
- **git push** — uploads the branch and its changes to the remote repository.

This workflow reduces the possibility of incomplete or faulty code breaking the main branch.

S3. HTML5 Input Types and Validation

For the application form, the appropriate HTML input types are:

- **Applicant Name:** Text
- **Email Address:** Email
- **Phone Number:** Telephone (tel)
- **Preferred Domain:** Select/drop-down list
- **Resume:** File upload

HTML5 provides built-in validation through attributes such as **required** and the **email input type**. The required attribute ensures that the user cannot submit the form without entering a value. The email type checks whether the entered value follows a basic email format.

However, HTML5 validation alone is not sufficient for production applications because client-side validation can be bypassed. **Server-side validation** is also required to verify and protect submitted data.

S4. Bootstrap Grid System

Bootstrap uses a **12-column responsive grid system**. Different grid classes determine how much space an element occupies at different screen sizes.

For three articles on desktop, each article can occupy **4 of the 12 columns**, because 12 divided by 4 gives 3 articles per row.

For mobile devices, each article can occupy all **12 columns**, causing one article to appear per row.

The Bootstrap classes used are:

col-12 col-lg-4

- **col-12** makes the article occupy the full width on smaller screens.
- **col-lg-4** makes the article occupy 4 columns on large screens and above.

Therefore, Bootstrap automatically changes the layout according to screen size without requiring custom media queries.

S5. Array and Two-Step Logic

An **array** is appropriate for storing rainfall data because all 30 readings are related and have the same type of data. An array stores them under one variable name and allows the program to access each reading using an index.

Using an array is much easier than creating 30 separate variables because loops can be used to process all readings efficiently.

The program follows two main steps:

1. **Calculate the average:** All 30 rainfall readings are added together, and the total is divided by 30.

2. **Find above-average days:** Each rainfall reading is compared with the calculated average. If a reading is greater than the average, its corresponding day is identified.

Two steps are required because the final average is not known until all 30 rainfall readings have been processed. The program therefore needs to calculate the average first and then perform the comparisons.

S6. Pointer and Empty String

A C string is a sequence of characters that ends with the **null character** `'\0'`. When the user enters an empty string, the string contains only the terminating null character.

A pointer used for string traversal must point to a valid memory location before it is dereferenced. Dereferencing an invalid or uninitialized pointer can cause **undefined behavior or a crash**.

The pointer-based solution should check that the pointer is valid before dereferencing it. It should also check for the null character so that traversal stops when the end of the string is reached.

String traversal can be performed in two ways:

- **Array indexing:** Uses an index such as `text[i]` to access characters.
- **Pointer traversal:** Uses a pointer such as `*ptr` and moves the pointer to the next character.

For beginners, **array indexing is generally easier to understand and debug**, because the position of each character is explicitly represented by an index.